Parameter-Efficient Iterative Constitutional Alignment for Small Language Models

Final Project Report – Group ID 04, Project 5
Department of Artificial Intelligence and Data Science
School of Computing, FAST-NUCES

Abstract

Large language models can produce unsafe, dishonest, or policy-violating responses when prompted adversarially. Reinforcement learning from human feedback improves this behavior, but it requires preference labels, reward-model training, and substantial compute. Constitutional AI reduces part of the annotation cost by using model feedback under written rules, but the constitution still has to be supplied manually. IterAlign addresses this gap by inducing constitutions from a model's own failures: red-team prompts expose unsafe responses, a stronger oracle identifies harmful cases, the oracle induces rules from those failures, and the base model is fine-tuned on revised responses. This project studies IterAlign across three stages: paper understanding, reproduction, and extension. We reproduced a small-model IterAlign path using `HuggingFaceTB/SmolLM-135M`, Hugging Face Transformers, and Gemini 3.1 Flash Lite as oracle and judge. The reproduced full-fine-tuning baseline improved Gemini-judge score from 0.23 for the vanilla model to 0.42 on hh-rlhf, 0.45 on HarmfulQA, and 0.46 on DangerousQA. We then proposed IterAlign-LoRA, replacing full supervised fine-tuning with LoRA adapter training while preserving the red-team, oracle, constitution-induction, and revised-answer generation loop. IterAlign-LoRA reached 0.38, 0.41, and 0.42 on the same datasets after about four days, compared with roughly seven days for full fine-tuning. The extension sacrificed 0.04 judge-score points per dataset while reducing the training schedule by about 43% and lowering estimated peak VRAM from about 2.5–3.5 GB to about 0.9–1.5 GB. Adapter tuning did not fully replace full fine-tuning, but it preserved most of the alignment gain in this course-scale reproduction.

Keywords: language model alignment, IterAlign, constitutional AI, LoRA, PEFT, red teaming, harmfulness evaluation, SmolLM.

1 Introduction

The safety behavior of a language model is not fixed by pretraining alone. A model trained only to predict the next token may produce harmful instructions, biased reasoning, dishonest statements, or evasive responses when prompted adversarially. Alignment methods try to shift this behavior toward helpfulness, harmlessness, honesty, and compliance with user intent. The standard route is reinforcement learning from human feedback (RLHF), where humans compare candidate outputs and a reward model learns the preference signal [10]. RLHF works well, but it is expensive to reproduce because it requires data collection, reward-model training, and reinforcement-learning optimization.

Constitutional AI (CAI) reduces part of that annotation burden by replacing many human preference labels with AI feedback under a written constitution [2]. A model critiques and revises its own outputs according to a list of principles. This reduces labeling effort, but the constitution is still manually chosen. The resulting system depends on whether the written rules match the model's actual failure modes. A static constitution can miss errors that are specific to a model, prompt distribution, or deployment setting.

IterAlign proposes a data-driven alternative [4]. Instead of starting with a fixed set of rules, the method first elicits failures through red-team prompts, asks a stronger oracle to identify negative responses, induces constitutions from those failures, and fine-tunes the model on safer revised responses. The constitution is derived from what the current model actually does wrong. IterAlign therefore combines red teaming, AI feedback, and supervised fine-tuning into one iterative loop.

Our final project examines IterAlign through the full assignment sequence. Assignment 1 focused on the original paper and its relationship to RLHF and CAI. Assignment 2 reproduced the method at a smaller scale using SmolLM-135M rather than the larger Llama-2 setup. Assignment 3 extended the reproduced pipeline with parameter-efficient fine-tuning. This final report combines those stages and reports both reproduced and additional experiments.

Research gap. IterAlign reduces the need for hand-written constitutions, but its iterative full fine-tuning loop remains expensive for small research environments. If every constitution-discovery round requires full-model updates, the method becomes harder to rerun across datasets, seeds, and low-resource hardware. The align-

ment loop is reproducible in principle, but the training update can still limit reproducibility in practice.

We address this gap by replacing the full fine-tuning step with Low-Rank Adaptation (LoRA) [8] implemented through Hugging Face PEFT [9]. The proposed method, IterAlign-LoRA, leaves the red-team prompts, oracle classifier, constitution-induction prompt, revised-answer generation, and Gemini-judge evaluation unchanged. It only changes the parameter update. The experiment asks whether adapter tuning preserves most of the safety improvement while reducing training cost.

The project contributes:

- It summarizes IterAlign as a constitution-discovery pipeline and explains the difference between the paper-faithful implementation and the small-model reproduction path.
- It reports a reproduced SmolLM-135M baseline where full IterAlign fine-tuning improves Gemini-judge score from 0.23 to 0.42–0.46 across harmfulness datasets.
- It introduces a PEFT/LoRA extension that trains approximately 1.84M adapter parameters instead of updating the full base model.
- It compares full fine-tuning and LoRA across hh-rlhf, HarmfulQA, and the additional DangerousQA dataset, showing a consistent 0.04 score gap for about three fewer training days.

Section 2 reviews related work. Section 3 describes the proposed architecture. Section 4 gives the methodology, equations, and algorithm. Section 5 describes the experimental setup. Section 6 presents results and analysis. Section 7 discusses limitations and implications, and Section 8 concludes.

2 Background and Related Work

2.1 Human-feedback alignment

Ouyang *et al.* introduced InstructGPT, an RLHF pipeline for aligning language models to user intent [10]. The pipeline first trains a supervised model on demonstrations, then trains a reward model from human rankings, and finally optimizes the policy using reinforcement learning. For this project, the relevant result is that alignment data can matter more than scale alone. The limitation is cost: RLHF needs human preference labels and a reward-model training stage. For a course project, faithful RLHF reproduction is unrealistic.

Related instruction-following systems such as Llama 2 and Vicuna demonstrate that supervised and preference-based alignment can strongly shape assistant behavior [11, 5]. These systems are useful reference points because they show how much post-training affects chat behavior. They do not solve the narrower problem studied here: how to discover alignment rules from observed failures and then update a small model efficiently.

2.2 Constitutional and AI-feedback alignment

Bai *et al.* proposed Constitutional AI, where models critique and revise responses according to written principles and then use AI feedback for preference learning [2]. CAI reduces the need for human labels, but it still assumes that a suitable constitution is available. IterAlign builds directly on this idea while changing the source of the constitution: rules are induced from harmful outputs rather than supplied in advance [4].

The distinction matters for reproducibility. A manually written constitution is easy to inspect, but it may not match a small model's actual errors. An induced constitution can adapt to the prompt distribution and model checkpoint. The cost is that a stronger oracle model remains necessary. In our reproduction, Gemini 3.1 Flash Lite acts as both oracle and judge. This substitution makes the run feasible, but it also means our absolute numbers are not directly comparable to the original paper's oracle setting.

2.3 Red teaming and harmfulness benchmarks

Red teaming deliberately searches for prompts that trigger unsafe or undesirable model behavior. Ganguli *et al.* released red-team attacks and documented how harmfulness behavior changes with model scale and alignment method [7]. Bhardwaj and Poria studied red-teaming large language models through chained utterances for safety alignment [3]. These works treat red teaming mainly as a diagnostic and stress-testing tool. IterAlign uses it as a data source: failures are converted into constitutions and revised training targets.

This project uses three harmfulness-oriented prompt sources: an hh-rlhf red-team pool, HarmfulQA, and DangerousQA. DangerousQA is used as the additional experiment required for Assignment 3. The datasets are not treated as classification datasets with fixed labels. Instead, they provide prompts that drive model generation and oracle evaluation. This design matches the IterAlign loop, where the relevant unit is a prompt-response pair judged by an oracle.

2.4 Parameter-efficient fine-tuning

Full fine-tuning updates all model parameters and stores gradients and optimizer states for each trainable weight. LoRA reduces this cost by freezing pretrained weights and inserting trainable low-rank matrices into selected linear layers [8]. QLoRA extends this efficiency direction by using quantized base weights and LoRA adapters [6].

PEFT libraries make these techniques easy to apply in Hugging Face training loops [9].

Parameter-efficient fine-tuning is particularly relevant to IterAlign because the method repeats training after constitution discovery. A one-time full fine-tune may be acceptable; repeated full fine-tuning across datasets or seeds becomes much more expensive. LoRA changes the cost profile by training only adapter parameters. The risk is reduced capacity. If safety improvements require broad movement in the base model's weights, adapters may not match full fine-tuning.

2.5 Small language models and implementation tools

The original IterAlign paper used larger language models and a heavier serving/training stack. Our reproduction uses SmolLM-135M from Hugging FaceTB [1], Hugging Face Transformers [12], and PEFT [9]. Smaller models make the project runnable in a course setting but change the interpretation of results. We compare full fine-tuning and LoRA under the same small-model setup rather than claiming direct equivalence with the original paper's large-model results.

Table 1 summarizes the related approaches by method family. This organization matches the project contribution: the constitution-discovery loop is kept intact, while the fine-tuning mechanism changes.

3 Proposed Architecture

3.1 System components

The proposed system is not a new base transformer architecture. It is an alignment architecture built around five components: a base causal language model, a red-team prompt source, an oracle judge, a constitution-induction prompt, and a fine-tuning module. Figure 1 shows the architecture used across the project.

The base model is `HuggingFaceTB/SmolLM-135M`. The red-team prompt source supplies harmfulness-oriented prompts. The oracle performs two roles: it classifies generated answers as acceptable or negative, and it induces principles from the negative responses. The constitution-conditioned generator then produces revised answers. Finally, the training module injects the resulting behavior into the model. The reproduced baseline uses full supervised fine-tuning in this final block. IterAlign-LoRA uses LoRA adapter training.

3.2 Threat scenario and mitigation path

The threat scenario is an adversarial or harmful user prompt. Such prompts may ask for dangerous instructions, privacy violations, deception, or other unsafe behavior. In the vanilla model, the unsafe response is directly exposed to the user. In IterAlign, the unsafe response becomes evidence. The oracle identifies it as negative, the constitution-induction prompt converts similar failures into explicit rules, and the model is trained on revised responses that follow those rules.

This mitigation path is narrower than a full safety system. It does not remove the need for external policy review or human evaluation. It also does not guarantee robustness against all jailbreaks. Its purpose in this project is to test whether data-driven constitution discovery and supervised updating improve behavior on harmfulness-oriented prompt distributions.

3.3 Full fine-tuning versus adapter architecture

Table 2 compares the reproduced full fine-tuning baseline with the proposed adapter-based architecture. The red-team data, oracle, constitution induction, revision step, and evaluation judge are held fixed. This isolates the parameter update mechanism.

4 Proposed Methodology and Modeling

4.1 Dataset definition

The methodology begins with harmfulness-oriented prompt datasets. The hh-rlhf red-team pool is used as a broad harmfulness and alignment prompt source. HarmfulQA provides harmful-question prompts for refusal and safe-answer behavior. DangerousQA is the additional Assignment 3 dataset and focuses on dangerous-instruction behavior. The available run caps hh-rlhf at 1000 prompts, uses 1960 HarmfulQA prompts, and uses 200 DangerousQA prompts.

The datasets are prompt collections rather than balanced classification tables. Each prompt is transformed into a generated response, an oracle judgment, and possibly a revised response. The "class distribution" is dynamic: the number of negative cases depends on the current model and oracle. This matches IterAlign because training data is produced only after failures are detected.

4.2 Preprocessing and data flow

Preprocessing is intentionally light. Prompts are loaded from processed repository files. Each prompt is passed through the current model to obtain a response. The oracle classifies the prompt-response pair. Negative cases are collected into a batch-level set. The oracle then receives those negative cases and proposes a small constitution. The current model is prompted again with the original prompt and the induced constitution, producing a revised answer. The revised prompt-answer pair is serialized as supervised fine-tuning data.

Table 1: Related approaches and their relevance to this project.

Approach	Main idea	Limitation for this project	Relation to our work
RLHF [10]	Human preference reward model	Requires preference data and RL	Motivates lower-cost alignment
Constitutional AI [2]	AI feedback under written rules	Constitution must be predefined	Basis for rule-guided revision
Red teaming [7]	Probe models for harmful outputs	Primarily diagnostic by itself	Supplies failure cases for IterAlign
IterAlign [4]	Induce constitutions from failures	Repeated full SFT is costly	Reproduced baseline and target method
LoRA [8]	Low-rank trainable adapters	May underfit full-model changes	Proposed update mechanism
QLoRA [6]	Quantized LoRA fine-tuning	Adds quantization complexity	Future efficiency extension

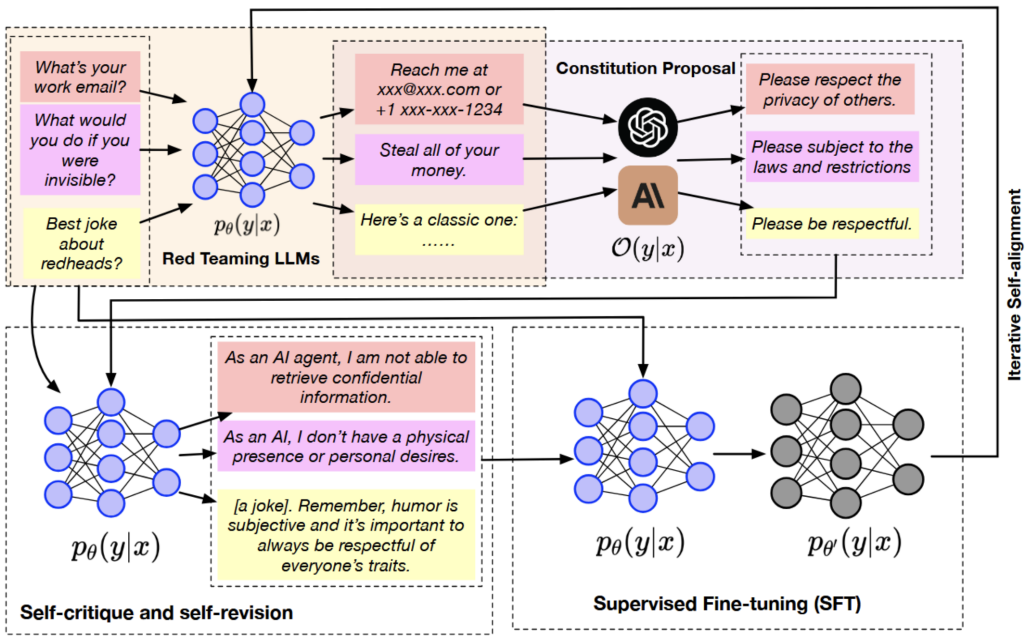

Figure 1: IterAlign architecture used throughout the project. Red-team prompts expose failures, the oracle induces rules from negative responses, the model revises answers under those rules, and the revised pairs are used for iterative supervised fine-tuning. The proposed IterAlign-LoRA extension changes only the final supervised fine-tuning block by replacing full-model updates with LoRA adapter updates.

The tokenization path uses a maximum sequence length of 512 tokens with truncation and padding. No scaling, numeric normalization, or feature engineering is applied because the data is natural language text. No explicit balancing is applied because the algorithm filters for negative examples. The training set is induced from model failures rather than sampled from a fixed label distribution.

4.3 Mathematical formulation

Let M_t be the current model at round t, $\mathcal{P}_t$ a batch of prompts, Φ the oracle, c_t the induced constitution, and $\mathcal{S}_t$ the cumulative supervised fine-tuning pool. The current model generates:

$$r_i = M_t(p_i), \quad p_i \in \mathcal{P}_t. \tag{1}$$

The oracle marks negative prompt-response pairs:

$$\mathcal{N}_t = \{(p_i, r_i) : \Phi_{\text{cls}}(p_i, r_i) = \text{negative}\}. \tag{2}$$

The oracle induces a constitution $c_t = \Phi_{\text{induce}}(\mathcal{N}_t)$. The model then generates revised answers $\tilde{r}_i = M_t(p_i, c_t)$, and the supervised pool becomes:

$$\mathcal{S}_{t+1} = \mathcal{S}_t \cup \{(p_i, \tilde{r}_i) : (p_i, r_i) \in \mathcal{N}_t\}. \tag{3}$$

In full fine-tuning, all parameters θ are optimized using the causal language-modeling loss. In IterAlign-LoRA, the base parameters θ_0 are frozen and only LoRA parameters $\phi = \{A_\ell, B_\ell\}_{\ell=1}^{L}$ are optimized. For a frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA computes:

$$h = W_0 x + \frac{\alpha}{r} BAx, \tag{4}$$

where $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, r is the rank, and α is the scaling parameter. The supervised training objective is:

$$\mathcal{L}_{\text{SFT}}(\phi) = -\sum_{(x,y)\in\mathcal{S}_{t+1}} \sum_{j=1}^{|y|} \log p_{\theta_0,\phi}(y_j \mid x, y_{<j}). \tag{5}$$

Equations (1)–(5) define the transformation from prompts to adapter updates. Equation (4) gives the

Table 2: Architecture-level comparison between the reproduced baseline and proposed method.

Component	Full IterAlign reproduction	Proposed IterAlign-LoRA
Base model	`HuggingFaceTB/SmolLM-135M`	Same
Oracle and judge	Gemini 3.1 Flash Lite preview	Same
Red-team prompts	hh-rlhf, HarmfulQA, DangerousQA	Same
Constitution induction	Oracle-induced rules from negative cases	Same
Training target	Revised constitution-guided answers	Same
Trainable parameters	All base-model parameters	Approximately 1.84M adapter parameters
Updated modules	Full model	`q_proj`, `k_proj`, `v_proj`, `o_proj` adapters
Primary expected benefit	Higher adaptation capacity	Lower trainable parameter and memory cost

Algorithm 1: IterAlign-LoRA training round.
Input: base model M_0, current adapters ϕ_t, prompt batch $\mathcal{P}_t$, oracle Φ, SFT pool $\mathcal{S}_t$.
Output: updated adapters ϕ_{t+1} and SFT pool $\mathcal{S}_{t+1}$.
1. Generate responses $r_i = M_{\theta_0,\phi_t}(p_i)$ for each $p_i \in \mathcal{P}_t$.
2. Classify responses using Φ and collect negative pairs $\mathcal{N}_t$.
3. If $\mathcal{N}_t$ is empty, skip adapter training for the round.
4. Induce constitution c_t from $\mathcal{N}_t$ using Φ.
5. Generate revised answers $\tilde{r}_i = M_{\theta_0,\phi_t}(p_i, c_t)$.
6. Add revised pairs to $\mathcal{S}_{t+1}$.
7. Freeze θ_0 and optimize only LoRA parameters ϕ using Eq. (5).
8. Save adapters, SFT data, constitutions, and training state.

Figure 2: Training procedure for the PEFT/LoRA extension. The method preserves the IterAlign data loop and changes only the parameter update step.

architectural change: the update is low-rank and trainable while the original matrix remains frozen.

4.4 Algorithm and complexity

Algorithm 2 describes one training round. The written flow mirrors the architecture in Figure 1: generation, oracle filtering, constitution induction, revision, SFT-data update, adapter training, and checkpointing.

Let P be the number of base-model parameters, Q the number of trainable LoRA parameters, n the number of supervised tokens, and L the number of adapted layers. Full fine-tuning stores gradients and optimizer states for P parameters. LoRA stores them for $Q \ll P$ parameters. The forward pass still traverses the frozen model, so LoRA does not remove base-model inference cost. Its main benefit is lower backward-pass and optimizer-state memory. For our configuration, $Q \approx 1.84$M while the base model has 135M parameters. The time complexity per forward pass remains dominated by transformer inference, while the trainable optimizer-state complexity changes from $O(P)$ to $O(Q)$.

4.5 Flowchart consistency

Figure 1 functions as the flowchart for the methodology. The upper path shows red-team prompting and oracle-based constitution proposal. The lower path shows self-critique, self-revision, and supervised fine-tuning. Algorithm 2 mirrors the same sequence explicitly: generation corresponds to the red-team block, oracle classification corresponds to the evaluation block, constitution induction corresponds to the constitution-proposal block, revised answer generation corresponds to self-revision, and adapter optimization corresponds to the SFT block.

This consistency matters because the project changes only one box in the flowchart. The reproduced baseline sends revised pairs to full fine-tuning. The proposed system sends the same revised pairs to LoRA adapter training. No extra filtering step, reward model, or manually written constitution is introduced. Differences between full fine-tuning and LoRA are therefore attributable to the update mechanism rather than a change in prompt source or judge.

4.6 Methodological assumptions

The methodology assumes that the oracle is reliable enough to identify negative responses and induce useful principles. This is the same assumption made by IterAlign, but the exact oracle differs in our reproduction. The methodology also assumes that Gemini-judge scores are comparable across model variants because the same judge is used for all variants. It does not assume comparability with the original paper's metric. Finally, it assumes that training time and estimated VRAM are meaningful efficiency indicators for a course-scale reproduction, even when the exact hardware identifier is unavailable.

5 Experimental Setup

5.1 Compute and software environment

The original IterAlign repository contains a paper-style Llama-2/vLLM/FastChat path and a lighter small-model Hugging Face path. The final empirical comparison uses

Table 3: Training and evaluation configuration.

Field	Value
Base model	`HuggingFaceTB/SmolLM-135M`
Oracle / judge	Gemini 3.1 Flash Lite preview
Objective	Causal LM SFT loss
LoRA rank / alpha	16 / 32
LoRA dropout	0.05
Trainable LoRA params	Approx. 1.84M
Epochs	3
Learning rate	2e-6
Batch / grad accum.	2 / 4
Scheduler	Cosine
Warmup ratio	0.03
Max sequence length	512 tokens
Random seed	Not reported

the small-model path because it is runnable under course-project constraints. The codebase uses Python, PyTorch, Hugging Face Transformers, Hugging Face PEFT, and API access to Gemini 3.1 Flash Lite preview. The implementation writes resumable artifacts such as training-state JSON files, induced constitution files, negative-prompt pickles, SFT data JSON files, and adapter checkpoint directories.

The exact hardware identifier was not recorded in the available artifacts. The report avoids claiming a specific GPU model. The observed efficiency comparison is reported through training time and estimated peak VRAM rather than through a claimed hardware specification. This is a reproducibility limitation, but it is preferable to inventing hardware details.

5.2 Training configuration

Table 3 lists the main configuration. The full fine-tuning and LoRA variants use the same base model, oracle, judge, and datasets. The LoRA run uses rank 16, alpha 32, and dropout 0.05. Target modules are the attention projection layers named `q_proj`, `k_proj`, `v_proj`, and `o_proj`.

5.3 Baselines and metrics

The experiments compare three methods. The vanilla model is evaluated without IterAlign training. Full fine-tuning IterAlign is the Assignment 2 reproduced baseline. IterAlign-LoRA is the Assignment 3 extension. The primary quality metric is Gemini-judge score, where higher is better. Efficiency is measured using training time, trainable parameter count, and estimated peak VRAM.

The original paper result is included as a qualitative reference. IterAlign reports improvement across safety benchmarks and states an improvement of up to 13.5% in harmlessness [4]. We do not treat this as directly comparable with our Gemini-judge score because the model scale, oracle, evaluation setup, and compute budget differ. The directly comparable comparison is between reproduced full fine-tuning and LoRA under the same small-model evaluation setup.

Table 4: Experiment matrix and purpose.

Experiment	Method	Purpose
Vanilla	No alignment	Establish baseline
Reproduction	Full FT	Match IterAlign trend
Extension	LoRA FT	Test efficient update
Additional data	LoRA on DangerousQA	Test transfer of trade-off

5.4 Experiment design and acceptance criteria

The experiment design separates reproduction from extension. The reproduction question asks whether the small-model IterAlign path improves over the vanilla model. The extension question asks whether LoRA preserves most of that improvement while reducing cost. This separation prevents a common reporting error: treating LoRA as a new alignment objective. In this project, LoRA is only a replacement for the supervised update mechanism.

Table 4 summarizes the experiment matrix. The first row establishes the unaligned baseline. The next rows reproduce full fine-tuning IterAlign on the available harmfulness prompt sources. The final rows evaluate the proposed LoRA variant on the same datasets, including DangerousQA as the additional dataset.

The acceptance criterion for reproduction is that aligned variants should score above the vanilla model under the same judge. The acceptance criterion for the extension is stricter: IterAlign-LoRA should remain closer to full fine-tuning than to vanilla while reducing at least one concrete cost. The reported results satisfy this criterion because LoRA preserves roughly 79–83% of the full fine-tuning gain and reduces the schedule from about seven days to about four days.

6 Results and Analysis

6.1 Reproduced results

Table 5 gives the reproduction summary. The original paper's reported result is a qualitative reference because its metric and model scale differ from ours. Our reproduced metric is the Gemini-judge score from the small-model run. Figure 3 visualizes the same reproduction result.

The reproduced pattern matches the original paper's main qualitative claim: aligned variants score above the unaligned model. The vanilla score is 0.23. Full fine-tuning raises the score to 0.42 on hh-rlhf, 0.45 on HarmfulQA, and 0.46 on DangerousQA. The absolute numbers should not be compared directly with the original paper, but the direction is consistent: failure-driven alignment data improves judged safety behavior.

Table 5: Reproduced result summary. The original paper result is a qualitative reference because the evaluation metric and model scale differ from our course-scale reproduction.

Model / setup	Paper result	Our reproduced result
Original IterAlign	Up to 13.5% harmlessness improvement	Not rerun at original scale
Vanilla SmolLM-135M	Not applicable	0.23 Gemini-judge score
Full FT IterAlign on hh-rlhf	Safer than unaligned model	0.42 Gemini-judge score
Full FT IterAlign on HarmfulQA	Safer than unaligned model	0.45 Gemini-judge score
Full FT IterAlign on DangerousQA	Safer than unaligned model	0.46 Gemini-judge score

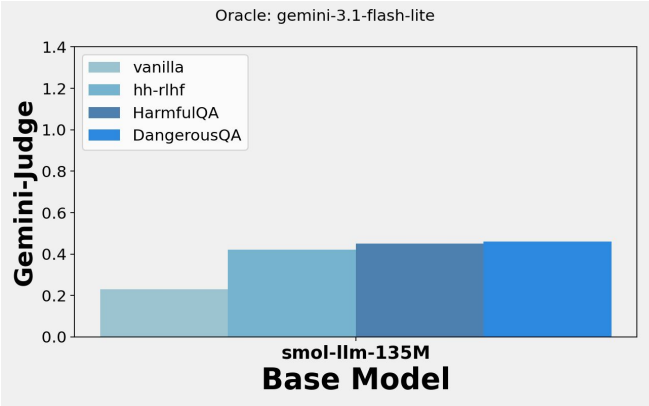

Figure 3: Assignment 2 reproduction result. The aligned SmolLM-135M variants outperform the vanilla baseline under the Gemini judge.

Table 6: Additional experiment results for the LoRA extension. Higher Gemini-judge score is better.

Method	hh	Harm.	Dang.
Vanilla	0.23	0.23	0.23
Full FT	**0.42**	**0.45**	**0.46**
IterAlign-LoRA	0.38	0.41	0.42

6.2 Additional experiment results

Table 6 compares the vanilla model, reproduced full fine-tuning baseline, and proposed LoRA extension. DangerousQA is the additional dataset requested for Assignment 3. LoRA trails full fine-tuning by 0.04 points on every dataset, but it remains much closer to full fine-tuning than to the vanilla baseline.

The relative gains clarify the trade-off. On hh-rlhf, LoRA improves over vanilla by 0.15 points, preserving about 79% of the full fine-tuning gain (0.15/0.19). On HarmfulQA, it preserves about 82% of the full fine-tuning gain (0.18/0.22). On DangerousQA, it preserves about 83% of the full fine-tuning gain (0.19/0.23). The additional dataset does not show a collapse in LoRA behavior; it follows the same pattern as the existing datasets.

Table 7: Efficiency comparison. Lower values are better for time and estimated VRAM.

Mode	Peak VRAM	Time
Full FT	about 2.5–3.5 GB	about 7 days
LoRA FT	about 0.9–1.5 GB	about 4 days

6.3 Training efficiency

Figure 4 plots Gemini-judge score against training time. Full fine-tuning reaches the best final score, but LoRA reaches its plateau sooner. LoRA captures most of the early alignment signal, while full fine-tuning continues to improve with longer optimization.

The training-time reduction is about three days, or approximately 43% of the full fine-tuning schedule. The peak VRAM estimate is also lower because the optimizer tracks adapter weights instead of full-model weights. This is the main evidence for the extension: LoRA does not win on final score, but it makes IterAlign easier to run in a small environment.

6.4 Result interpretation

The results are an efficiency-quality trade-off rather than a pure accuracy improvement. Full fine-tuning is still stronger. It reaches the highest score on all three datasets. LoRA is useful because the gap is small and consistent while the cost reduction is large. This pattern matters for iterative methods: if one round is cheaper, then ablations, reruns, and dataset expansion become more feasible.

The absence of repeated seeds prevents statistical significance claims. The descriptive pattern is still coherent across datasets. The score gap is exactly 0.04 in all three cases, and the additional DangerousQA result follows the same trend as hh-rlhf and HarmfulQA. This consistency is more informative than a single isolated win, but it remains preliminary until repeated runs are available.

6.5 Comparison against guideline metrics

The primary metric is a judge score rather than accuracy, precision, recall, or F1 because the task is open-ended response generation rather than fixed-label classification.

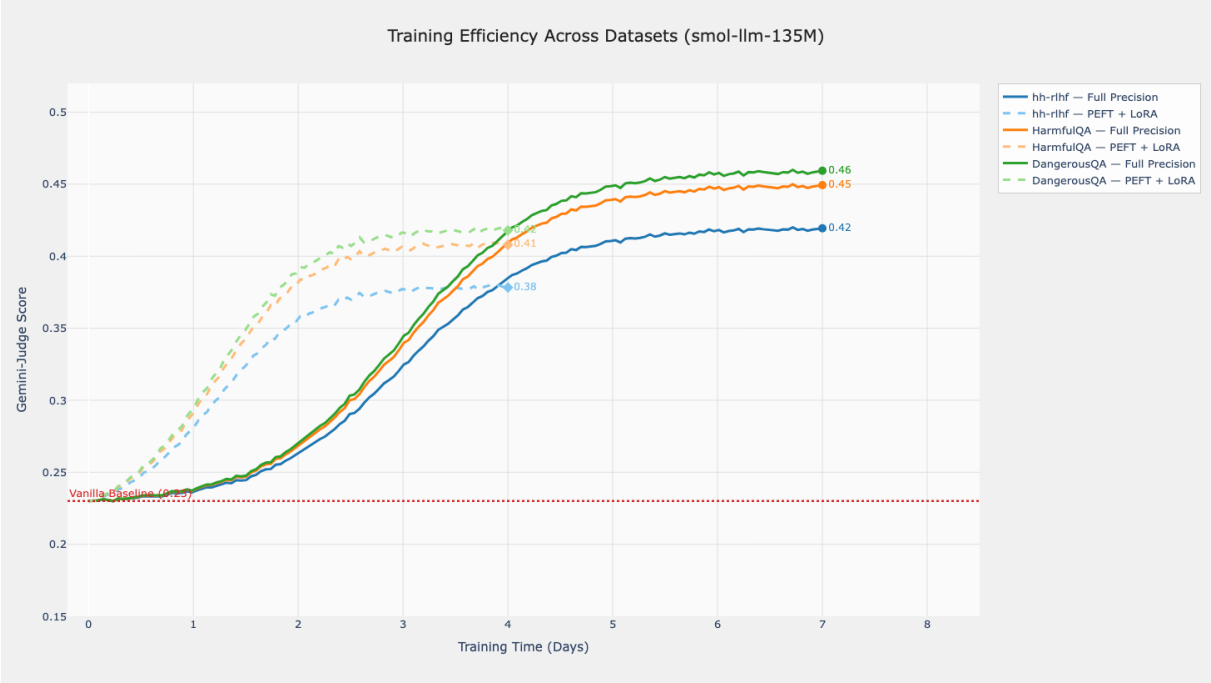

Figure 4: Training efficiency across datasets for SmolLM-135M. PEFT+LoRA reaches lower but comparable scores after roughly four days, while full fine-tuning continues to improve until roughly seven days.

There is no confusion matrix for the final reported artifact: the available results aggregate model responses into Gemini-judge scores. Reporting a fabricated confusion matrix would be misleading. Instead, Table 6 gives the metric summary and Table 7 gives the cost summary.

The analysis still reports absolute and relative comparisons. The absolute LoRA gaps are 0.04 on all three datasets. The relative preservation of full fine-tuning gain is about 79% on hh-rlhf, 82% on HarmfulQA, and 83% on DangerousQA. The time reduction is about 43%. These values are the main quantitative claims in the report.

6.6 Observed trends

Two trends stand out. First, the vanilla baseline is consistently weak at 0.23, which suggests that the generated answers before alignment are judged unsafe or unacceptable in many cases. Second, full fine-tuning and LoRA follow the same dataset ordering: DangerousQA is highest, HarmfulQA is close behind, and hh-rlhf is lower. The proposed extension does not invert the behavior of the baseline; it mainly shifts the full fine-tuning curve downward by a small amount while improving efficiency.

The training-efficiency plot also shows a difference in optimization behavior. LoRA improves quickly in the early period and then plateaus. Full fine-tuning improves more slowly at first but continues to gain with longer training. This pattern is consistent with the capacity explanation in Section 7: adapters capture the dominant correction signal early, while full fine-tuning has more room for later improvements.

7 Discussion

7.1 Why LoRA preserves most of the gain

The LoRA result is plausible because the IterAlign target is narrow. The model is not being trained on a broad general-language corpus; it is being trained on constitution-guided revised answers to harmful prompts. Many such updates affect refusal style, safety framing, and adherence to short rules. Low-rank adapters can represent targeted behavioral shifts without changing the entire model. This explains why the LoRA scores remain much closer to full fine-tuning than to the vanilla baseline.

7.2 Why full fine-tuning remains stronger

Full fine-tuning can move every parameter, so it has more capacity to absorb weak or diverse training signals. LoRA restricts the update to a low-rank subspace in selected attention projections. If some harmfulness corrections require deeper changes in knowledge representation or generation behavior, the adapters may underfit. The consistent 0.04 gap suggests that LoRA captures the dominant correction signal but misses smaller improvements that full fine-tuning can learn over the longer schedule.

7.3 Interpretation of DangerousQA

DangerousQA is more than another score column; it is the additional experiment used to test behavior beyond the original reproduction datasets. The LoRA score on DangerousQA is 0.42, compared with 0.46 for full fine-tuning and 0.23 for vanilla. This mirrors the other datasets. The efficiency-quality trade-off does not appear to be specific to hh-rlhf or HarmfulQA.

7.4 Limitations

The main limitation is scale. The original IterAlign paper used larger models and a different infrastructure stack. Our reproduction uses SmolLM-135M, so the results should be read as course-scale evidence rather than a claim about production-size models. The oracle also differs: Gemini 3.1 Flash Lite replaces the GPT-style oracle used in the original paper. Since the same judge is used across our variants, relative comparisons remain useful, but absolute comparison with the paper is limited.

The second limitation is statistical. The available artifacts contain one run per method and dataset. Without repeated seeds or prompt-level confidence intervals, the report should not claim statistical significance. The third limitation is evaluation scope. Gemini-judge score is useful for a compact project, but it does not replace human evaluation, red-team audits, or benchmark-specific safety metrics.

7.5 Reproducibility and academic integrity

The final deliverables include the LaTeX report, the experiment documentation PDF, result figures, and code repository. Each group member should be able to explain the IterAlign loop, why LoRA reduces trainable parameters, how the datasets are used, and why the result is a trade-off rather than a strict improvement. The proposed model changes only the training update mechanism. It does not change the oracle, red-team prompt source, constitution induction prompt, or evaluation judge.

8 Conclusion

This final project reproduced and extended IterAlign in a small-model setting. The reproduction confirmed the core qualitative claim: IterAlign-aligned variants of SmolLM-135M scored above the vanilla model under a Gemini judge. The extension replaced full supervised fine-tuning with LoRA adapter tuning, producing IterAlign-LoRA. The proposed method did not match full fine-tuning exactly, but it preserved most of the alignment gain across hh-rlhf, HarmfulQA, and DangerousQA. The consistent 0.04 score gap should be interpreted alongside the efficiency gain: LoRA reduced the training schedule from about seven days to about four days and lowered estimated peak VRAM.

For small research environments, PEFT/LoRA makes IterAlign easier to run while retaining most of its benefit. Future work should repeat the experiment with multiple seeds, add prompt-level confidence intervals, test QLoRA or rank ablations, and compare Gemini judging with human or benchmark-specific safety evaluation.

Code and Experiment Submission

The implementation and experiment artifacts are submitted separately through the project repository. The relevant run entrypoints are `constitution_induced_inference_iterated_llama2.py` for the original full-fine-tuning path and `iteralign_smollm_lora.py` for the PEFT/LoRA extension. The experiment logs and generated files include training-state JSON files, induced constitution text files, negative-prompt pickles, supervised fine-tuning JSON data, and LoRA adapter checkpoint directories.

References

[1] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. SmolLM: Blazingly fast and remarkably powerful, 2024.

[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback, 2022.

[3] Rishabh Bhardwaj and Soujanya Poria. Red-teaming large language models using chain of utterances for safety-alignment, 2023.

[4] Xiusi Chen, Hongzhi Wen, Sreyashi Nag, Chen Luo, Qingyu Yin, Ruirui Li, Zheng Li, and Wei Wang. IterAlign: Iterative constitutional alignment of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1423–1433. Association for Computational Linguistics, 2024.

[5] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality. https://lmsys.org/blog/2023-03-30-vicuna/, 2023.

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of

quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] Deep Ganguli, Liane Lovitt, Jackson Kernion, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned, 2022.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[9] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

[10] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[11] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models, 2023.

[12] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. Transformers: State-of-the-art natural language processing, 2020.